BANCO DE DADOS

Prof. Nathan Saraiva

@prof_nathan.saraiva

nathan.saraiva@ifpi.edu.br

SUMÁRIO

- Introdução a Banco de Dados
- Arquitetura de SGBDs
- Modelos de Dados
- Modelo Entidade-Relacionamento
- Modelo Relacional
- Dependência Funcional
- Normalização
- Algebra Relacional
- Linguagem SQL

Introdução a Banco de Dados

Valor dos Dados

1

Dados Brutos

Fatos isolados sem contexto - números, texto, datas que sozinhos não têm significado claro.
Representa algo não tem sentido a princípio.

2

Informação

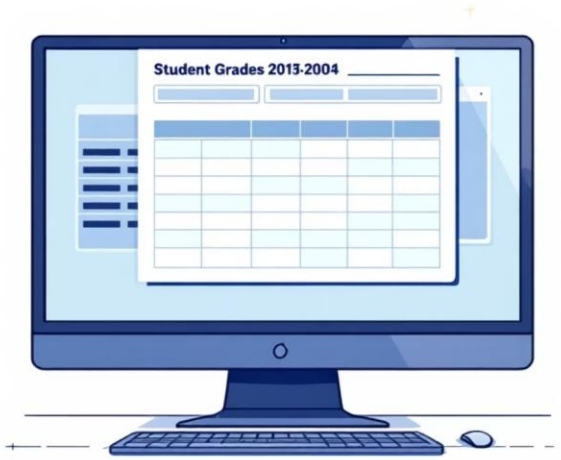
Dados organizados e processados que ganham contexto e relevância para tomada de decisões.
Dados tratados ou processados ou compreendidos.

3

Conhecimento

Informações analisadas e interpretadas que geram insights para ações estratégicas.
Conhecimento vai além de informações

Exemplo Prático



Dado

João: 8.5

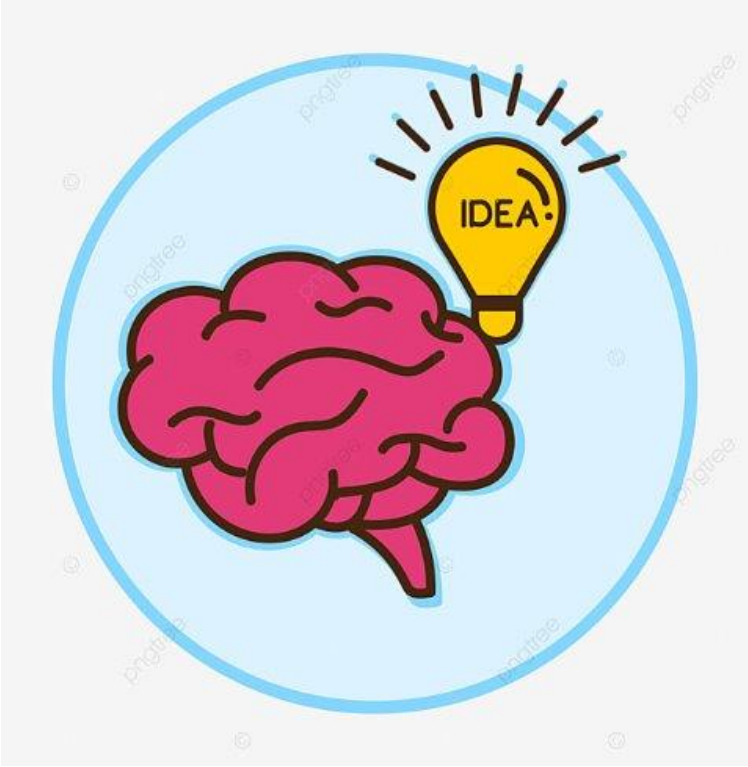
Informação

João obteve nota 8.5 em Matemática

Conhecimento

João tem bom desempenho e pode ajudar colegas com dificuldades.

Sabedoria



- A Análise correta do conhecimento pode ainda gerar **SABEDORIA**.
- **Correta** tomada de decisões baseadas em conhecimento
- Aplicação do conhecimento, ou seja, quais ações podem ser tomadas através do conhecimento adquirido.

Wisdom
SABEDORIA
Habilidade de tomar decisões bem informadas e agir com eficácia
com base na compreensão do conhecimento subjacente.
+ Insight
Knowledge
CONHECIMENTO
Resultado da análise e interpretação
da informação para descobrir padrões, tendências e relações.
Fornece entendimento sobre "como" e "por que" certo fenômeno ocorre.
+ Significado
Information
INFORMAÇÃO
Dados organizados, estruturados e contextualizados.
É útil para responder perguntas básicas como "quem", "o quê",
"onde" e "quando".
+ Contexto
Data
DADOS
Fatos discretos e objetivos sobre um evento.
É o fundamento das etapas subsequentes, mas tem valor limitado isoladamente.
Valor / Utilidade
Maturidade / Inteligência

Problemas dos Arquivos Tradicionais

Redundância de Dados

Mesmas informações armazenadas em múltiplos locais, causando causando inconsistências e desperdício de espaço.

Dificuldade de Acesso

Buscar informações específicas exige Identificação correta e tempo excessivo para localização

Isolamento dos Dados

Informações espalhadas em diferentes formatos impedem análises integradas e relatórios completos

Integridade Comprometida

Ausência de controles automáticos permite dados incorretos ou contraditórios no sistema

Mais Problemas do Modelo Tradicional

Problemas de Concorrência

Múltiplos usuários não conseguem acessar simultaneamente sem conflitos

Segurança Limitada

Controle básico de acesso permite violações de privacidade e dados sensíveis

Estes problemas motivaram o desenvolvimento de soluções mais robustas e organizadas.

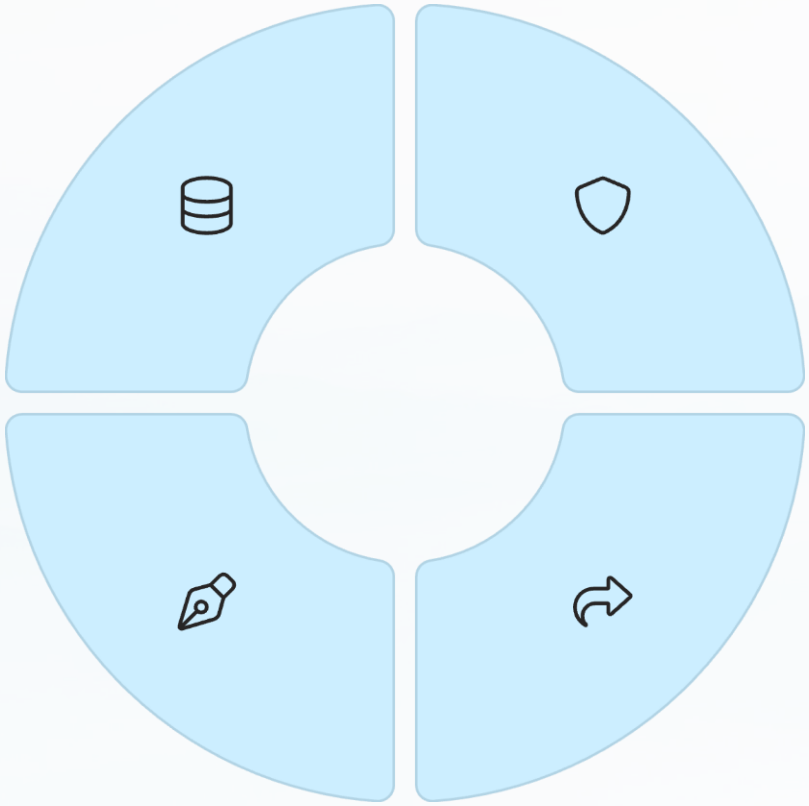
O que é um Banco de Dados?

Coleção Organizada

Dados estruturados e relacionados armazenados sistematicamente

Acesso Controlado

Permissões e segurança para diferentes tipos de usuários



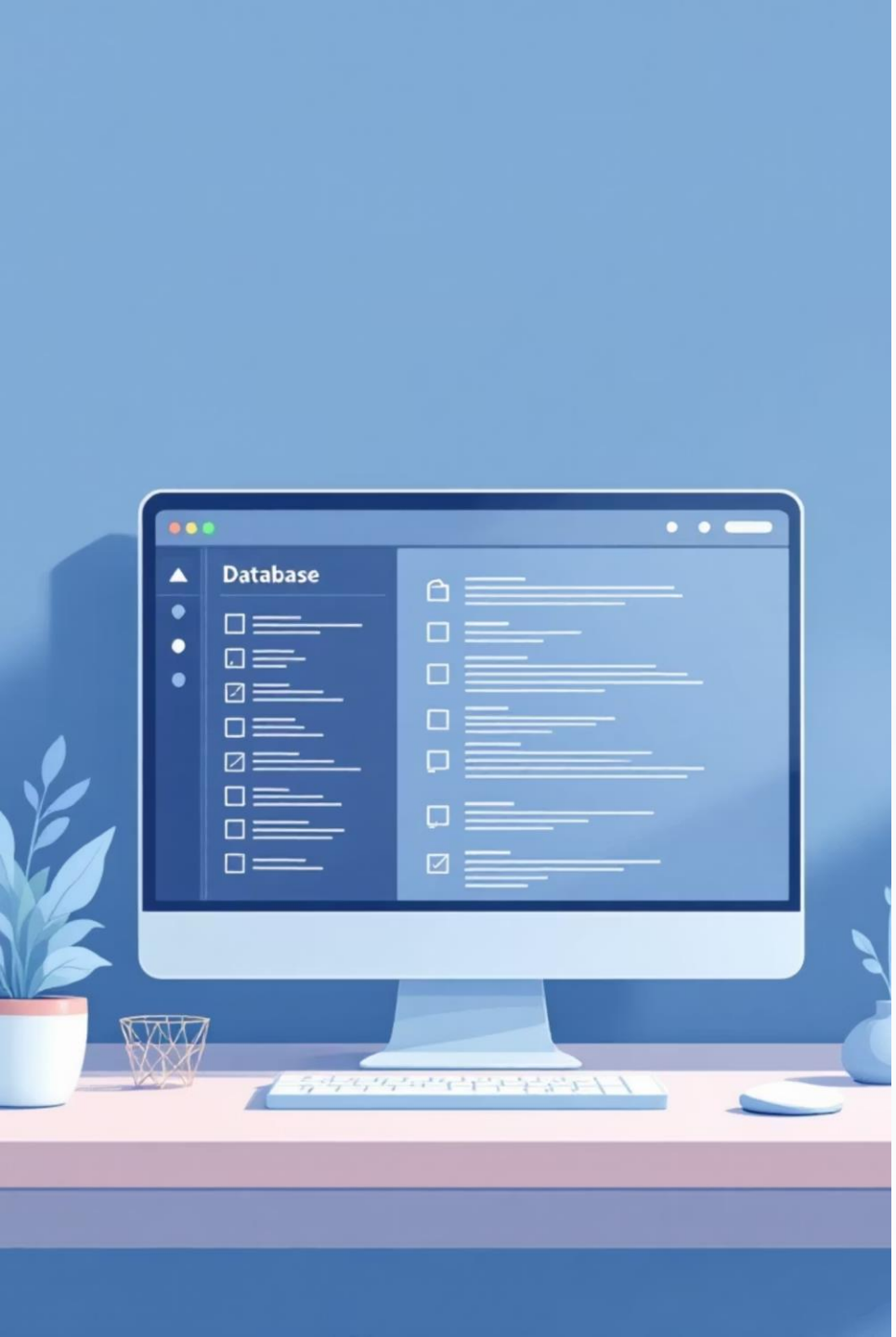
Integridade

Regras garantem consistência e qualidade qualidade dos dados

Compartilhamento

Múltiplos usuários e aplicações podem acessar simultaneamente

Um banco de dados elimina os problemas dos arquivos tradicionais através de organização inteligente.



E o SGBD?

O **Sistema de Gerenciamento de Banco de Dados (SGBD)** é o software que:

Cria e Define

Estabelece estruturas e relacionamentos entre os dados

Manipula

Permite inserir, atualizar, excluir e consultar informações

Controla

Gerencia acesso, segurança e integridade dos dados

Vantagens dos SGBDs

Redução da Redundância

Elimina duplicação desnecessária de dados

Compartilhamento

Acesso simultâneo e seguro por múltiplos usuários

Backup e Recuperação

Proteção automática contra perda de dados

Padrões e Flexibilidade

Estruturas padronizadas que se adaptam às mudanças

Limitações dos SGBDs

Complexidade

Requer conhecimento técnico especializado para implementação e manutenção adequadas

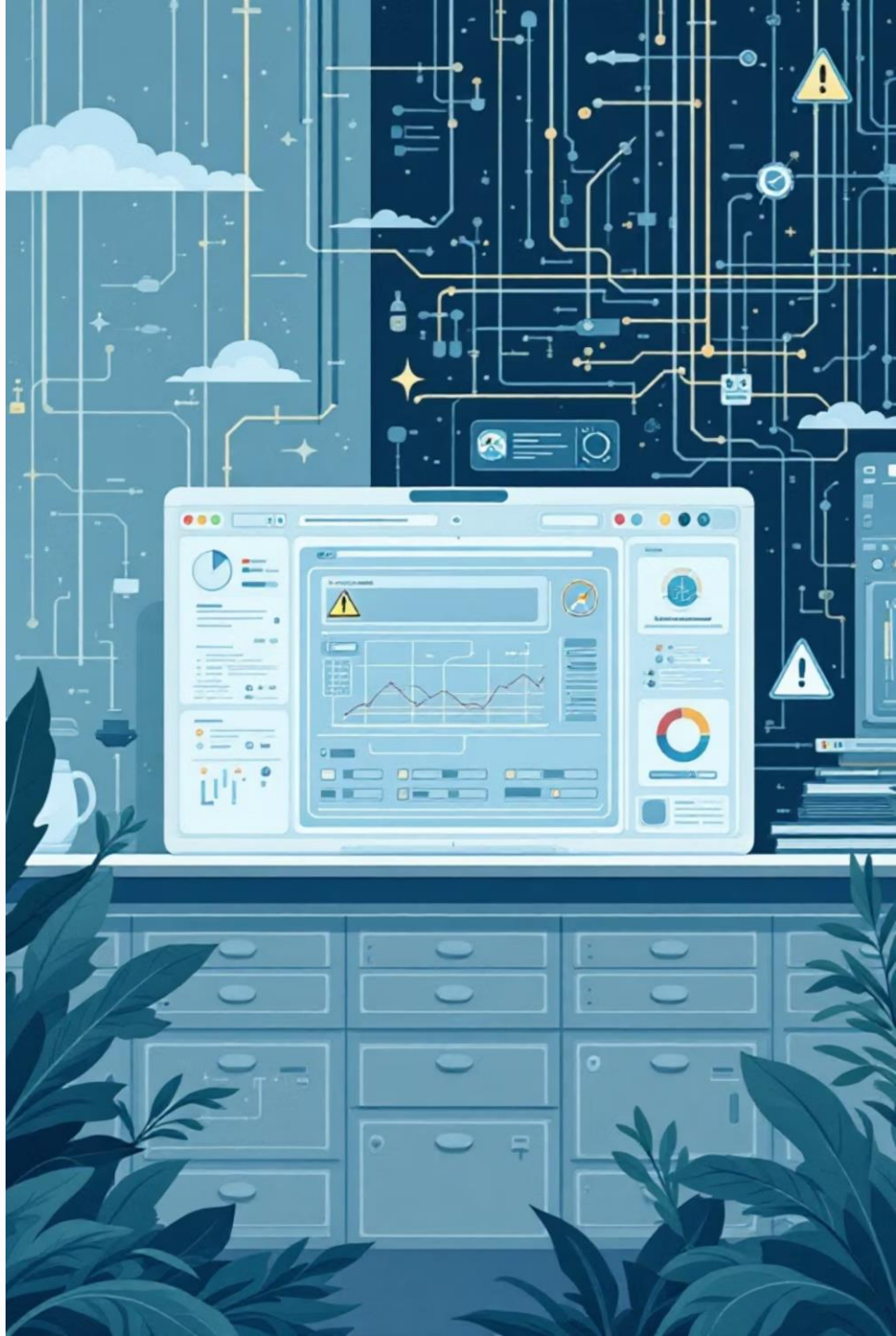
Custo Elevado

Investimento significativo em software, hardware e treinamento treinamento da equipe técnica técnica

Sobrecarga para Aplicações Simples

Pode ser excessivo para sistemas pequenos com poucos dados e usuários usuários

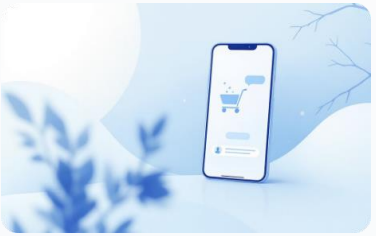
ⓘ Importante avaliar custo-benefício antes da implementação



SGBDs no Seu Dia a Dia

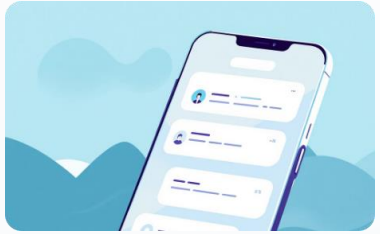
Bancos

Suas transações, saldo e histórico financeiro são gerenciados por SGBDs robustos



E-commerce

Produtos, pedidos, estoque e dados de clientes em plataformas como Amazon



Redes Sociais

Posts, comentários, conexões e preferências no Facebook, Instagram e Twitter

Sistemas Acadêmicos

Notas, matrículas, histórico escolar e dados dos estudantes estudantes

Os SGBDs são invisíveis mas essenciais - eles organizam e protegem as informações que usamos todos os dias!

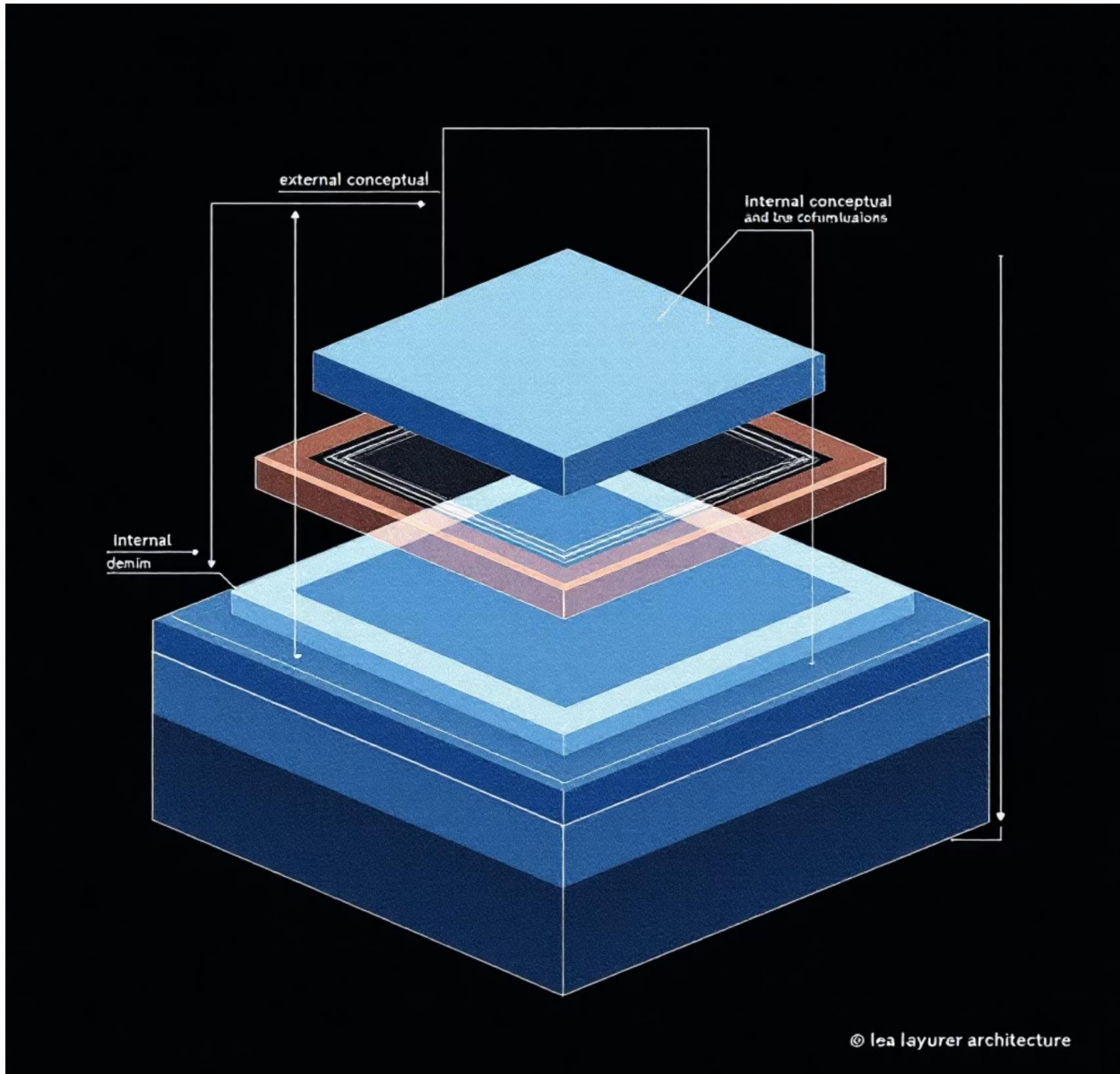
ARQUITETURA DE SISTEMA DE BANCO DE DADOS

Introdução à Arquitetura ANSI/SPARC

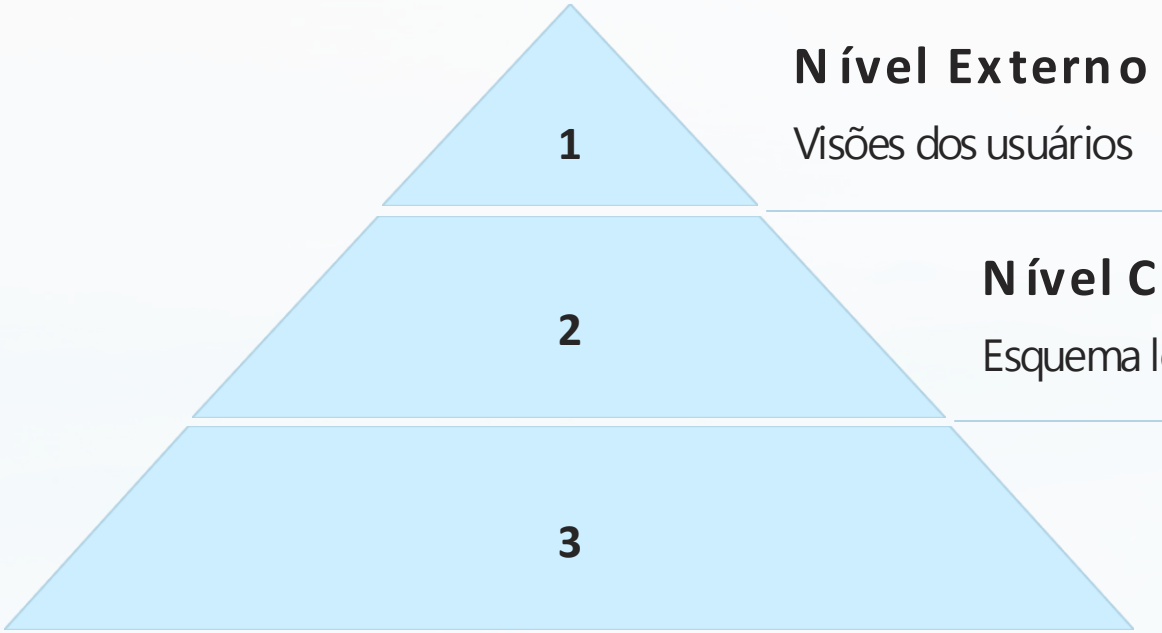
A arquitetura ANSI/SPARC, desenvolvida pelo comitê ANSI (American National Standards Institute) e pelo grupo SPARC (Standards Planning and Requirements Committee), é um modelo conceitual fundamental que define como os sistemas de banco de dados devem ser estruturados.

Esta arquitetura estabelece **três** níveis distintos de abstração, cada um com responsabilidades específicas, criando uma separação clara entre diferentes aspectos do sistema de banco de dados.

O modelo ANSI/SPARC é essencial para compreender como os SGBDs modernos organizam e gerenciam dados de forma eficiente e flexível.



Os Três Níveis da Arquitetura ANSI/SPARC



Nível Externo

Visões dos usuários

Nível Conceitual

Esquema lógico global

Nível Interno

Armazenamento físico

Cada nível da arquitetura possui esquemas específicos que definem a estrutura e organização dos dados em diferentes perspectivas, desde a visão do usuário final até o armazenamento físico no disco.

Detalhamento dos Níveis

Nível Externo (Visão)

Define diferentes visões dos dados para usuários específicos. Cada usuário ou grupo de usuários vê apenas os dados relevantes para suas necessidades, ocultando complexidade desnecessária e mantendo segurança através do princípio do menor privilégio.

Nível Conceitual (Lógico)

Representa a estrutura lógica completa do banco de dados. Define entidades, relacionamentos, restrições de integridade e regras de negócio. É independente de considerações físicas de armazenamento e serve como base para todas as visões externas. Visão do DBA.

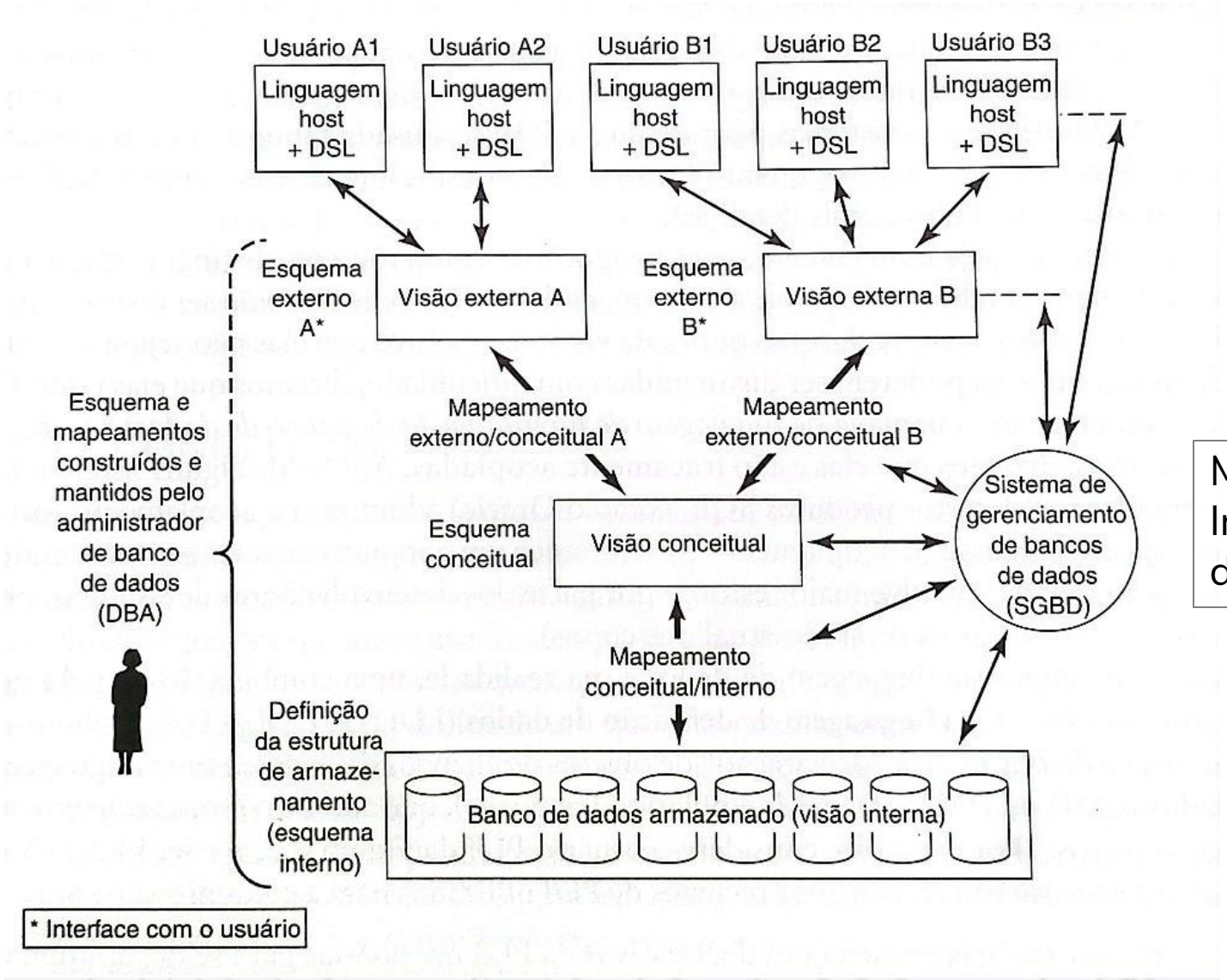
Nível Interno (Físico)

Especifica como os dados são fisicamente armazenados e acessados. Inclui estruturas de indexação, métodos de acesso, organização de arquivos e otimizações de performance específicas do hardware e sistema operacional.

COMPARATIVO

Nível / camada	Nome alternativo	O que representa / função	Quem interage / visibilidade	Responsabilidade chave
Externo	Visão de usuário / external	Conjunto de **visões** dos dados que diferentes usuários ou aplicações veem	Usuários / aplicações	Permitir visões personalizadas, segurança (ocultar dados irrelevantes) GeeksforGeeks +3
Conceitual	Lógico / global	Modelo global de dados: entidades, relacionamentos, restrições — sem preocupação com armazenamento físico	Administrador de banco de dados / modelo global	Integrar visões externas, definir regras de integridade e estrutura lógica dos dados Scribd +3
Interno	Físico / armazenamento	Representação física: estruturas de armazenamento, índices, organização de disco, compressão etc.	Sistema (SGBD) + sistemas de armazenamento	Gerenciar como os dados são armazenados e acessados eficientemente no hardware TutorialsPoint +3

ARQUITETURA DETALHADA DO SISTEMA



Mapeamento: Independência de dados lógica e física

Arquitetura do PostgreSQL

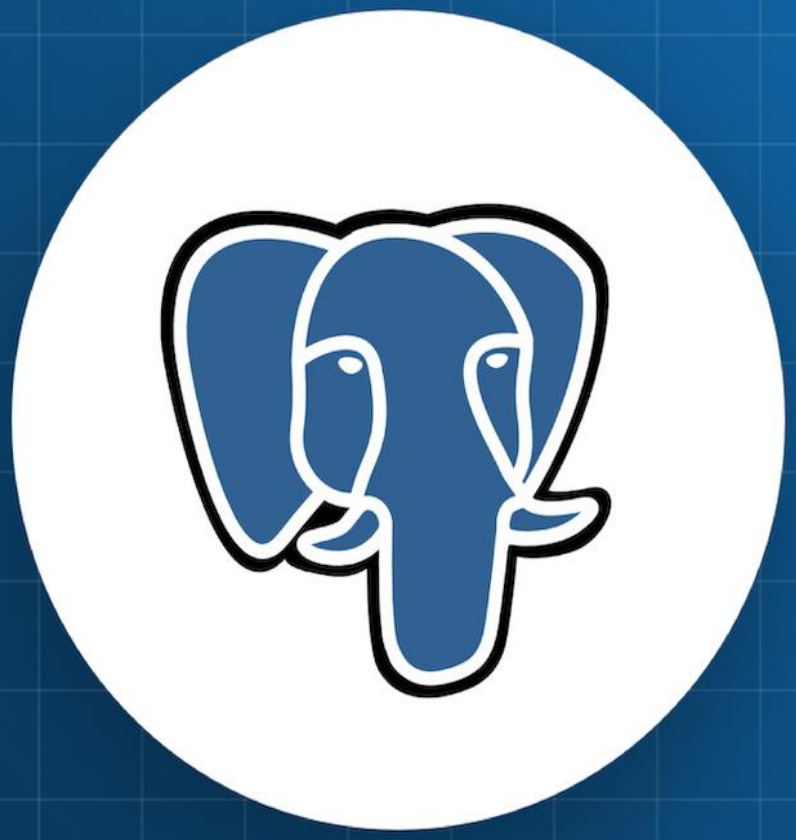
Componentes Principais

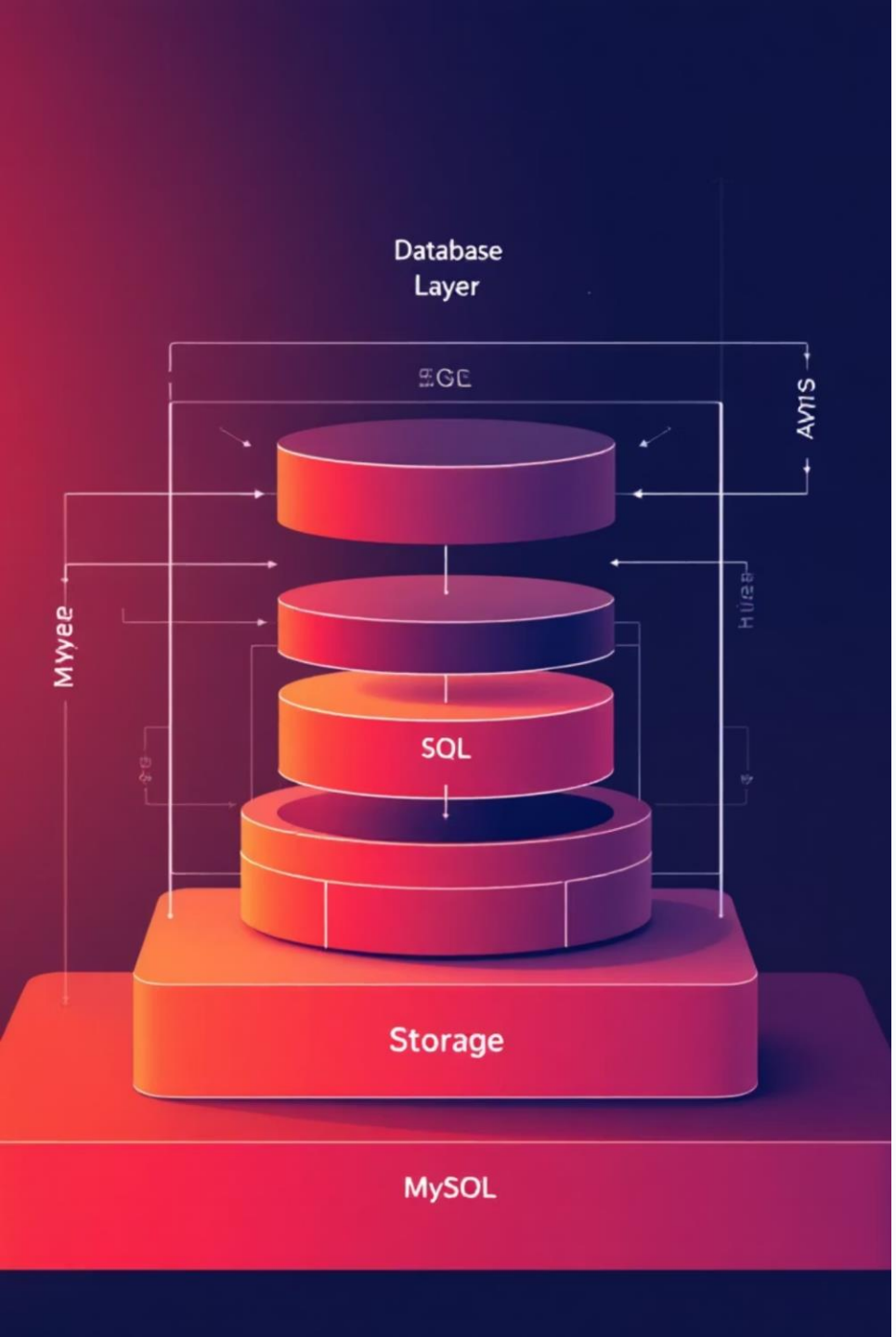
- **Postmaster:** Processo principal que gerencia conexões e processos backend.
- **Background:** Vários processos que atuam para manter o BD funcionando
- **Backend Processes:** Um processo por conexão de cliente, executa comandos SQL
- **Shared Memory:** Área compartilhada usado por todos os processos
- **WAL Logs:** Escreve logs de transação

Características Técnicas

O PostgreSQL implementa a arquitetura ANSI/SPARC. Sistema cliente/servidor.

Utiliza arquitetura multi-processo com memória compartilhada, proporcionando isolamento entre conexões e estabilidade do sistema.





Arquitetura do MySQL

Connection Layer

Gerencia autenticação, threads de conexão e segurança. Cada conexão cliente recebe uma thread dedicada.

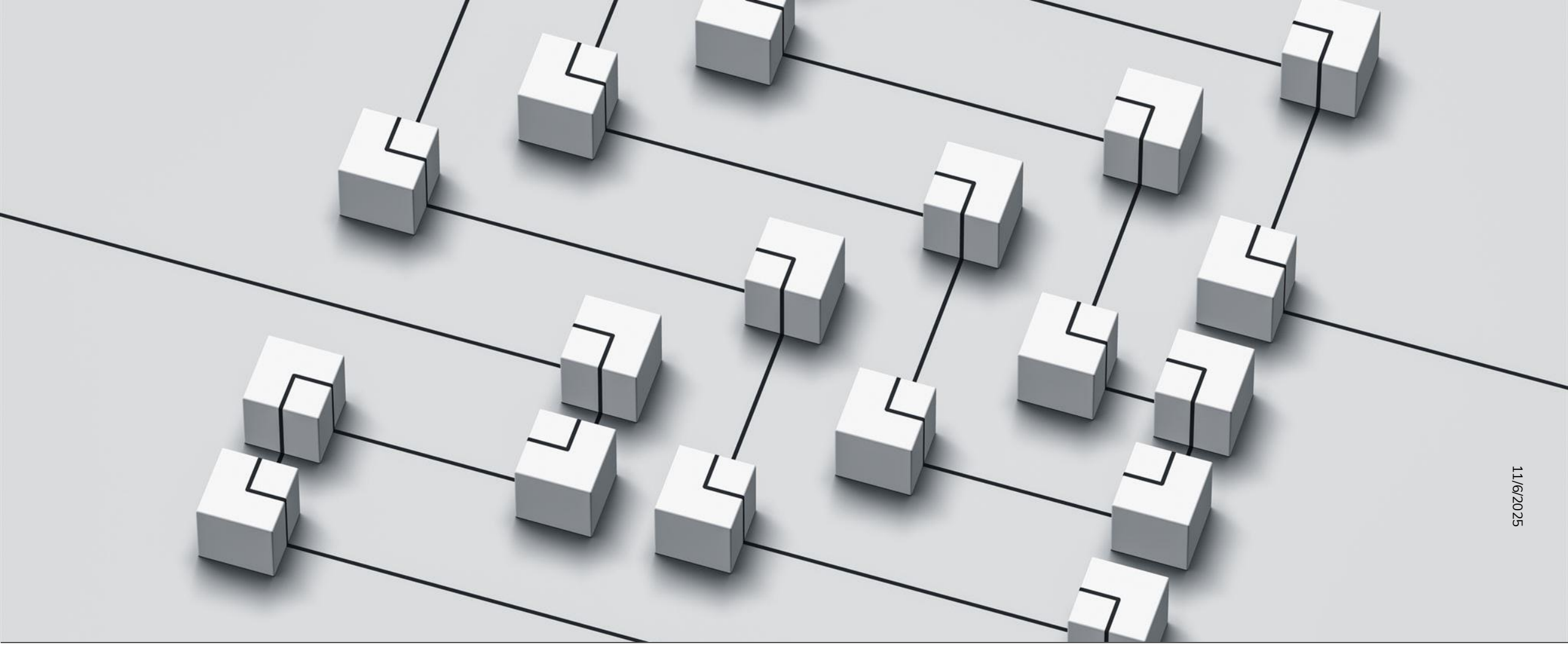
SQL Layer

Parser, otimizador de consultas, cache de cache de consultas e execução de comandos SQL. Independente do mecanismo de armazenamento.

Storage Engine

Camada plugável (InnoDB, MyISAM, Memory) que gerencia armazenamento físico, indexação e transações e se comunica com os dados

A arquitetura modular do MySQL é baseada em um modelo cliente-servidor, onde um servidor centraliza os dados e as funcionalidades, e os clientes (aplicações ou usuários) enviam requisições SQL para acessar ou manipular esses dados.



MODELAGEM DE DADOS

MODELAGEM DE DADOS

- Definição: Processo de criar representações visuais de sistemas de informação.
  - Representação visual dos dados e suas relações.
  - Serve como “mapa” para o desenvolvimento de sistemas e bancos de dados.
  - Facilita comunicação entre analistas, desenvolvedores e gestores.
  - Ajuda a identificar inconsistências e redundâncias nos dados.
- Importância: Ajuda a organizar dados, entender requisitos e facilitar o design de bancos de dados.

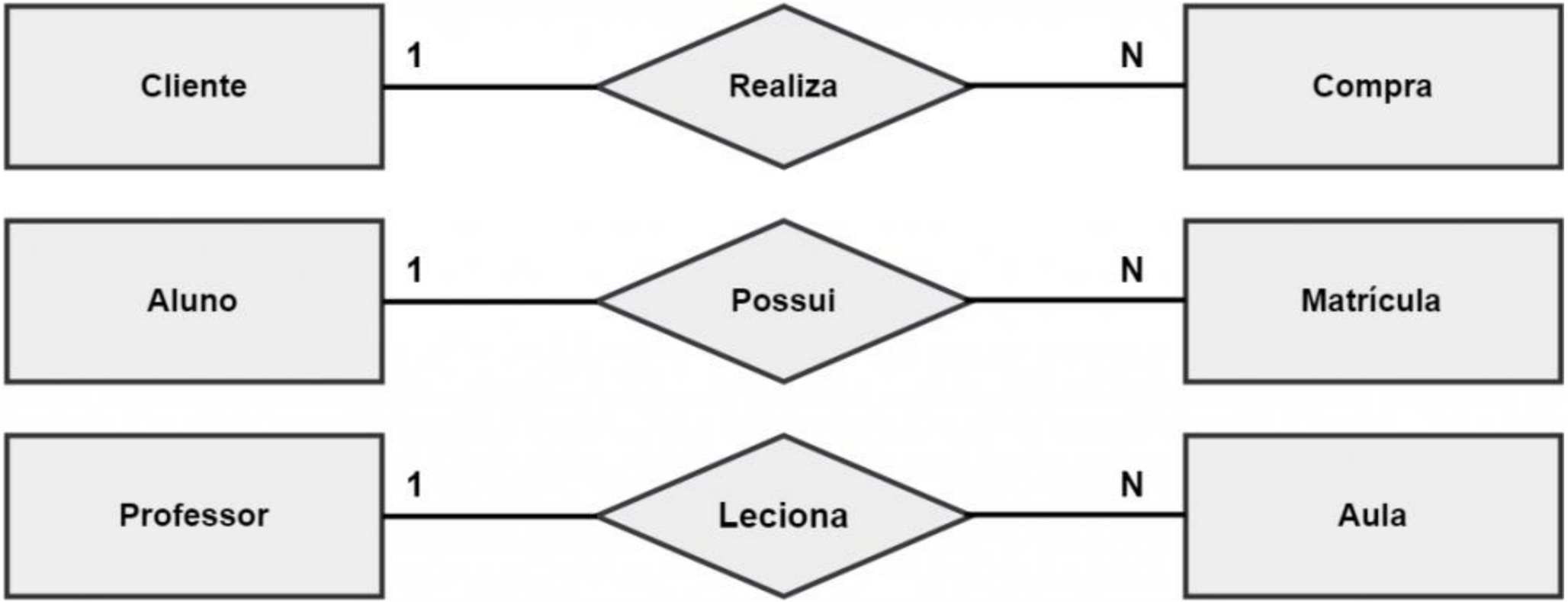
BENEFÍCIOS DA MODELAGEM DE DADOS

- **Melhor compreensão do negócio**: traduz regras de negócio em estruturas de dados.
- **Redução de erros**: evita inconsistências e redundâncias.
- **Facilidade na manutenção**: alterações futuras são mais simples.
- **Suporte à integração de sistemas**: dados bem modelados facilitam integração com outros sistemas.

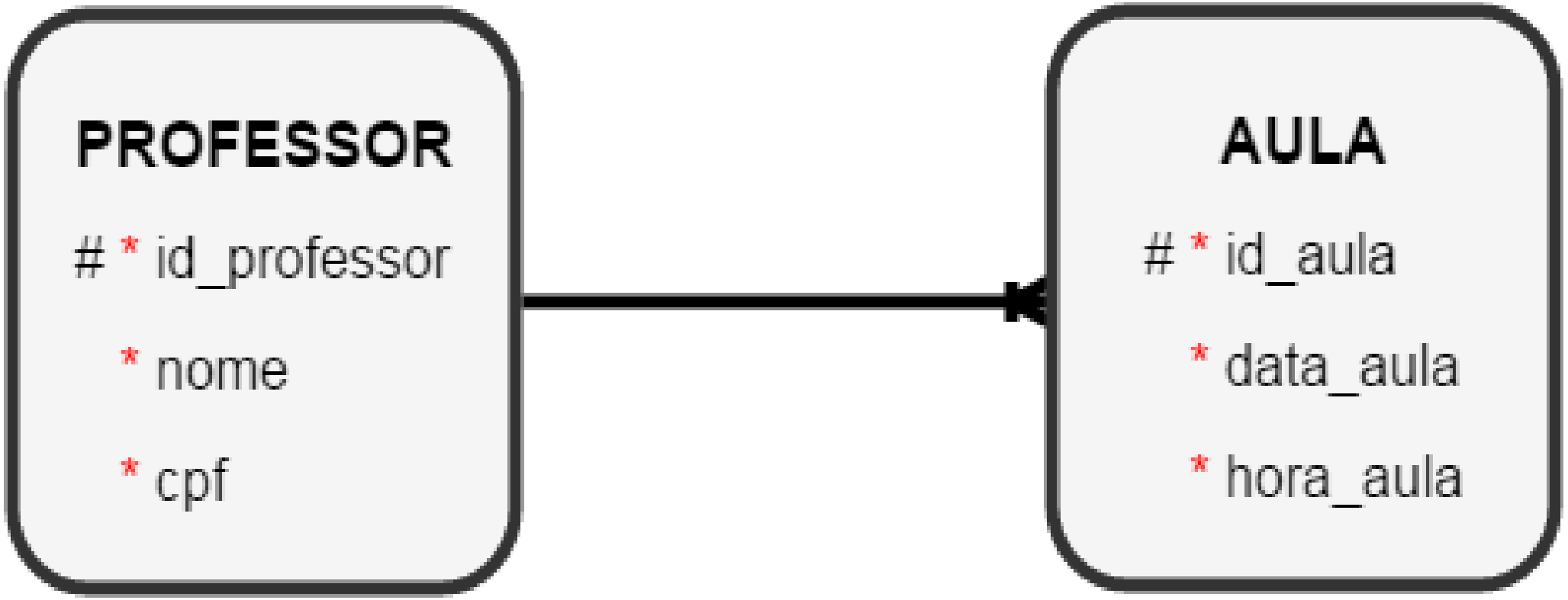
TIPOS DE MODELOS DE DADOS

- **Modelo Conceitual**
  - Visão de alto nível, focada em regras de negócio.
  - Ex.: Diagrama Entidade-Relacionamento (DER).
- **Modelo Lógico**
  - Detalha entidades, atributos e relacionamentos.
  - Independente do SGBD.
- **Modelo Físico**
  - Representação implementável no SGBD.
  - Define tabelas, índices, chaves e tipos de dados.

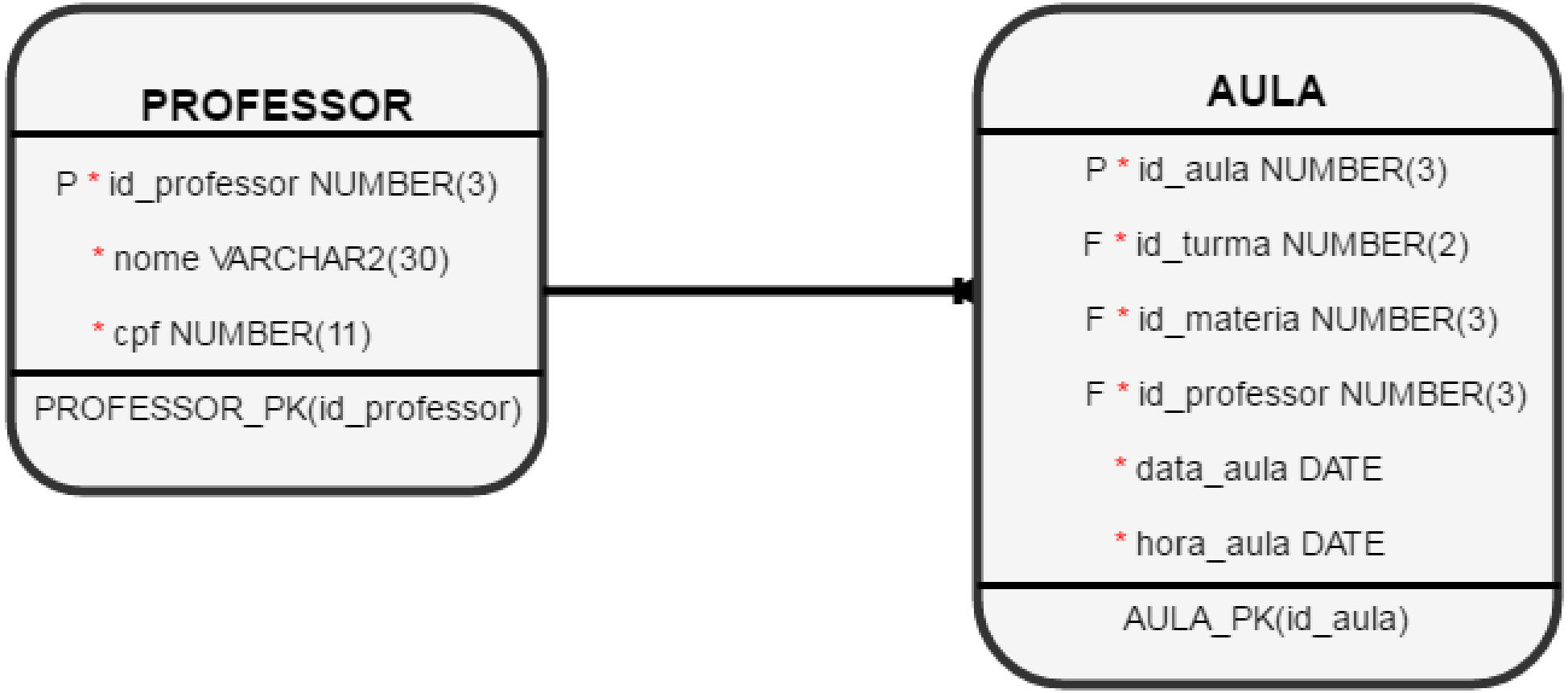
MODELO CONCEITUAL



MODELO LÓGICO

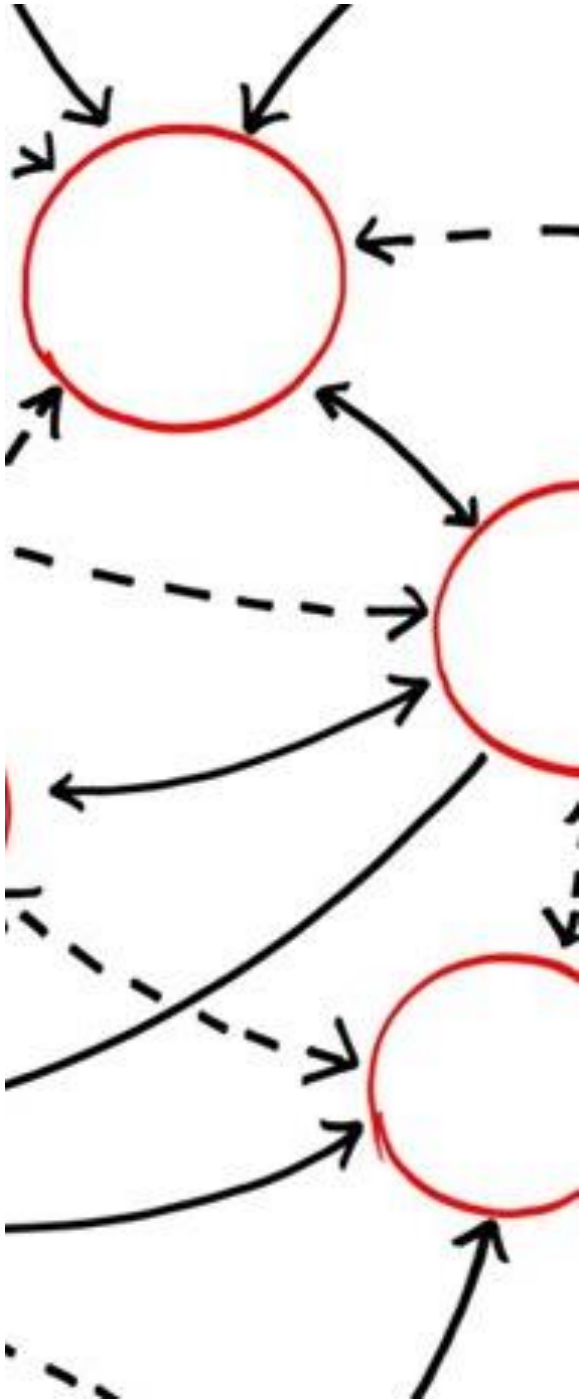


MODELO FÍSICO



PROCESSO DE MODELAGEM DE DADOS

1. Identificação de entidades principais.
2. Definição de atributos essenciais.
3. Determinação de relacionamentos entre entidades.
4. Normalização e definição de chaves primárias/estrangeiras.
5. Refinamento e validação do modelo.
6. Implementação no banco de dados.



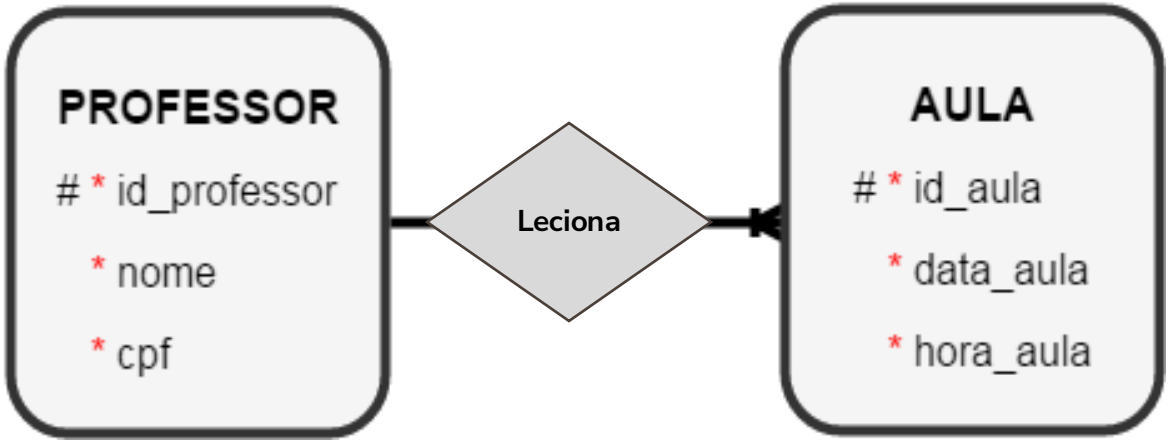
MODELOS DE BANCO DE DADOS

- **Hierárquico**: dados organizados em árvore (pai → filho).
  - Ex.: Organização da empresa → departamentos → funcionários.
- **Em Rede**: múltiplos relacionamentos entre dados (grafo).
  - Ex.: Funcionários participando de vários projetos.
- **Relacional**: dados em tabelas com linhas (tuplas) e colunas (atributos).
  - Suporta SQL e garante integridade com chaves.
- **Orientado a Objeto**: objetos complexos armazenados no banco
- **NoSQL**: Dados não estruturados e semiestruturados como documentos, gráficos e dados em cache
  - Ex: MongoDB, Cassandra e Neo4J
- **Em memória**: Acesso ultrarápido atraves de acesso via memoria principal
  - Ex: Redis e Memcached
- **Em nuvem**: Banco de dados hospedados em nuvem
  - Ex: Amazon RDS, Azure SQL Database
- **Distribuído:** Dados distribuídos e imutáveis
  - Tecnologia Blockchain usado em criptomoedas

MODELO ENTIDADE-RELACIONAMENTO

MODELO ENTIDADE-RELACIONAMENTO (MER)

- **Conceito**
  - Representação conceitual utilizado na Engenharia de Software para descrever os objetos (**entidades**) envolvidos em um domínio de negócios, com suas características (**atributos**) e como elas se relacionam entre si (**relacionamentos**).
- **Histórico**
  - Peter Chen desenvolveu a modelagem ER na década de 1970



OBJETIVOS DO MER

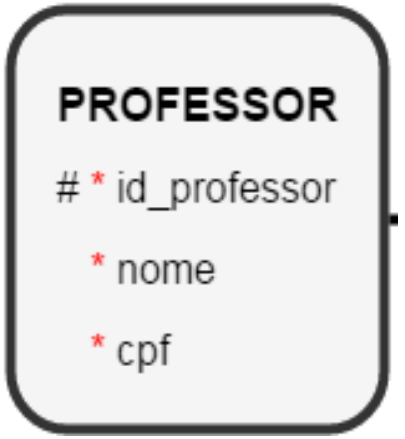
- Descrever a estrutura lógica de um banco de dados de forma visual e abstrata, facilitando o projeto de bando de dados

- Facilitar a comunicação entre analistas, desenvolvedores e usuários.

- Servir como base para a construção do modelo lógico e físico do banco de dados.

ENTIDADES

- Representam objetos do mundo real ou conceitos sobre os quais queremos armazenar informações. Possuem uma existência independente.
  - Ex.: Aluno, Curso, Professor, Diretor, Turma, Aula.
- Pode existir fisicamente.
  - Ex.: Aluno, Professor, Diretor.
- Pode ser um objeto de existência conceitual.
  - Ex.: Curso, Turma, Coordenação, Aula
- As entidades possuem atributos, que são propriedades que a descrevem.
  - Ex.: uma entidade Funcionario pode ser descrita por atributos como nome, data de nascimento, endereço, matricula.

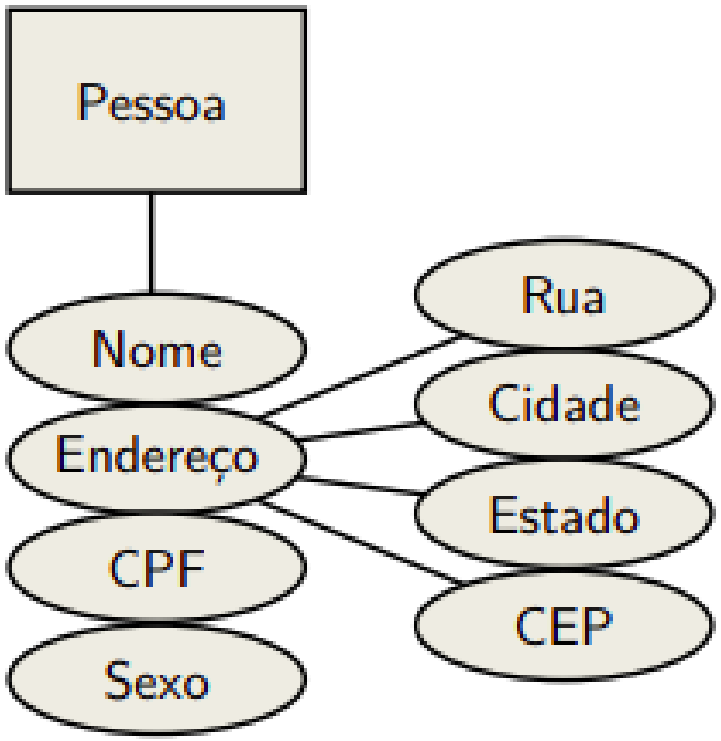


TIPOS DE ENTIDADES

- Entidades Fortes
  - Existência independe de outras entidades
  - Ex.: Em um sistema de vendas, a entidade **Produto** independe de qualquer outra.
- Entidades Fracas
  - Dependem de outras entidades para existirem
  - Ex.: Entidade **Venda** depende da entidade produto
- Entidades Associativas
  - Surge quando há a necessidade de associar uma entidade a um relacionamento existente
  - Ex.: Entidade **Brinde** relacionado com um processo de venda, não está relacionada nem com **Venda** e nem com o **Produto** mas sim com o relacionamento entre as duas entidades

ATRIBUTOS

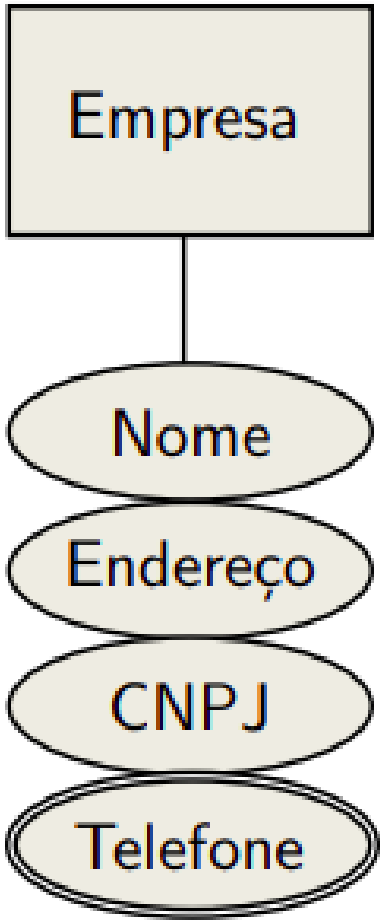
- Características que descrevem cada entidade
- Classificação quanto a sua estrutura:
  - **Simples**
    - Não pode ser dividido em partes menores.
    - Ex.: CPF, Sexo, Peso
  - **Composto**
    - Podem ser divididos em partes menores, que representam outros atributos
    - Ex.: Endereço



TIPOS DE ATRIBUTOS

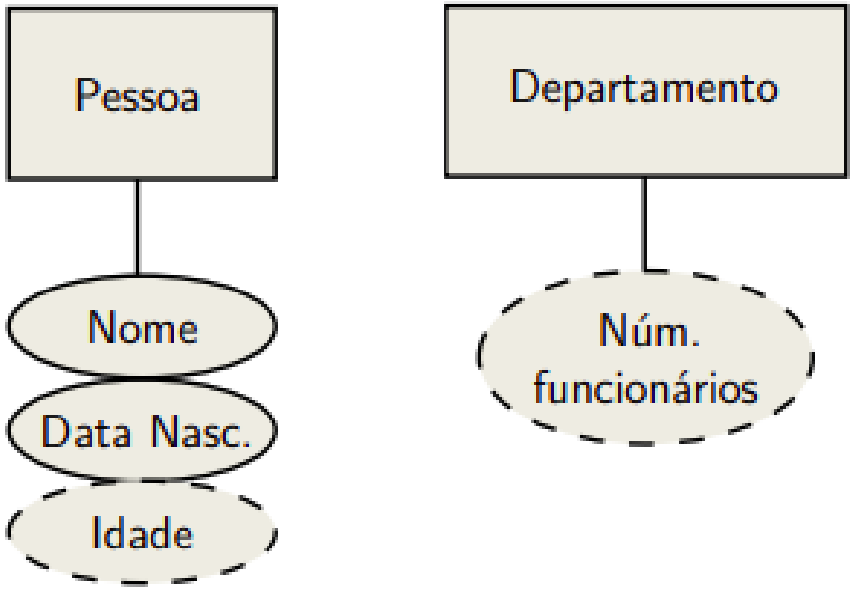
- Quanto a quantidade de valores:
  - **Monovalorado**
    - Possui um único valor para uma entidade em particular
    - Exemplos: Nome, CNPJ
  - **Multivalorado**
    - Possui um conjunto de valores para cada entidade.
    - Ex.: Graduação, Dependentes (Pessoa)

      Telefone (Empresa)



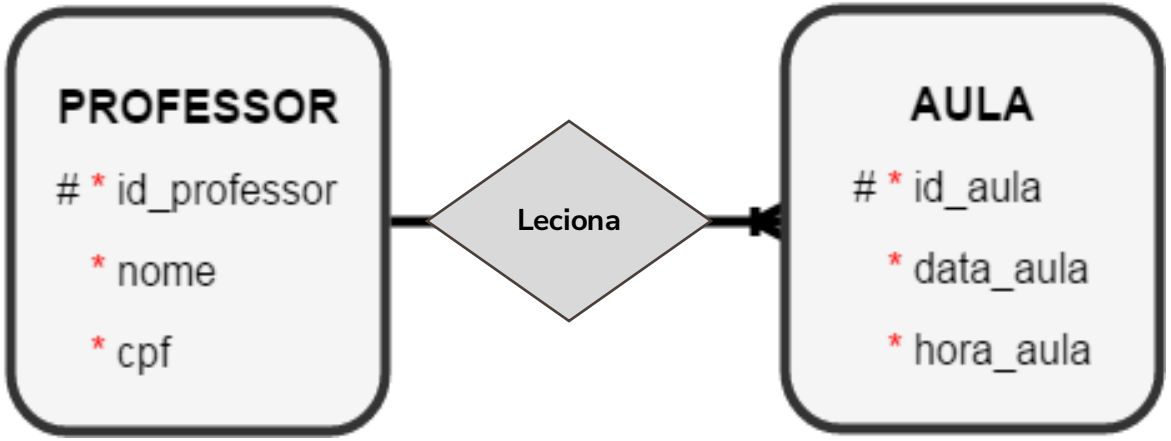
TIPOS DE ATRIBUTOS

- Quanto ao armazenamento no BD:
  - **Armazenado**
    - Atributo da entidade realmente armazenado no BD
  - **Derivado**
    - Refere-se ao atributo cujo valor pode ser obtido a partir dos valores de outros.
    - Ex1.: idade de uma pessoa pode ser calculada pela data de nascimento
    - Ex2.: determinar o número de empregados de um departamento



RELACIONAMENTOS

- Como entidades atuam umas sobre as outras ou estão associadas uma com a outra
- Representado por verbos
- Ex1.: Uma pessoa **trabalha** para uma empresa
- Ex2.: Professor leciona uma aula



TIPOS DE RELACIONAMENTO

- UM PARA UM (1:1)
  - Onde uma entidade X se associa unicamente a uma ocorrência da entidade Y
  - Ex.: Aluno --> Matricula --> Curso

TIPOS DE RELACIONAMENTO

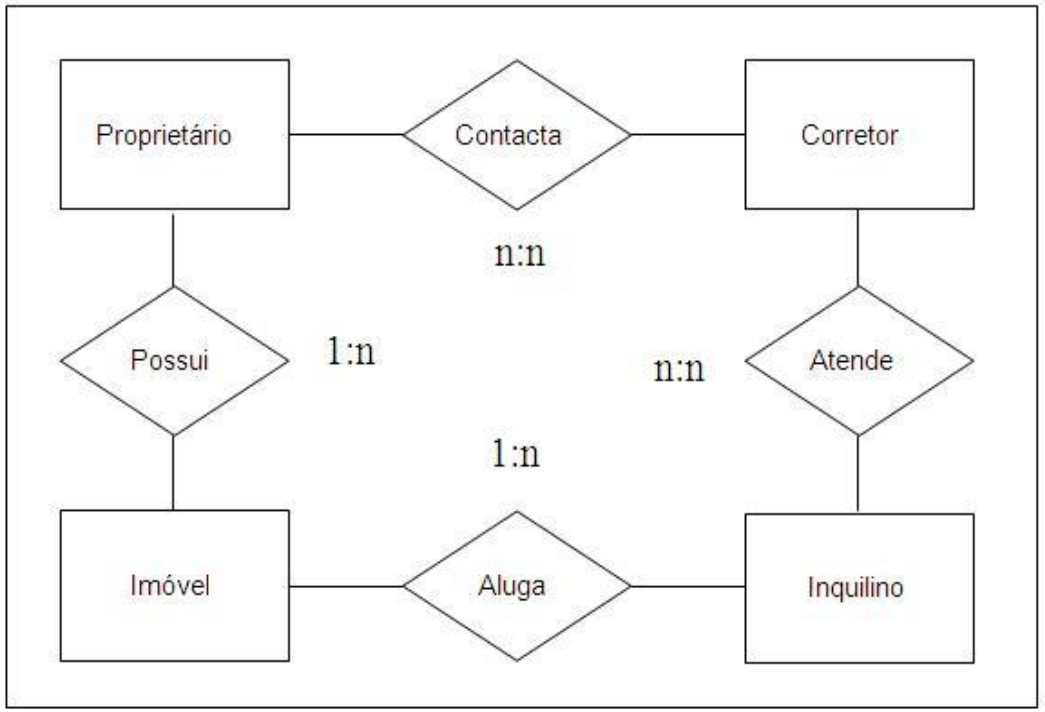
- UM PARA MUITOS (1:N)
  - Onde uma entidade X se associa a várias ocorrências da entidade Y, porém, a entidade Y pode apenas se associar a uma ocorrência da entidade X.
  - Ex.: Departamento --> emprega --> N funcionários (N = {0,1,...})

TIPOS DE RELACIONAMENTO

- MUITOS PARA MUITOS (N:N)
  - Onde a entidade X o pode se associar a várias ocorrências da entidade Y e a entidade Y pode também se associar a várias ocorrências da entidade X.
  - Ex.: N Autores --> escreve --> N livros

CARDINALIDADE

- Define o número mínimo e máximo de ocorrências de uma entidade em um relacionamento.
- Exemplo:
  - Um curso **pode ter muitos alunos** → cardinalidade 1:N.
- Notações comuns:
  - (0,1), (1,1), (0,N), (1,N)



MODELO RELACIONAL

INTRODUÇÃO AO MODELO RELACIONAL

- O modelo relacional foi proposto por **E. F. Codd** em 1970.
- Baseia-se em **matemática relacional** (teoria dos conjuntos).
- Utiliza **tabelas (relações)** para representar dados e suas inter-relações.
- É o modelo mais utilizado em bancos de dados atualmente.

CONCEITOS FUNDAMENTAIS

- **Relação**: Tabela com linhas e colunas. Geralmente representa um tipo de entidade do mundo real, como "Clientes" ou "Produtos".
- **Tupla**: Linha da tabela (registro). Representa uma única ocorrência de uma entidade na tabela
- **Atributo**: Coluna da tabela (campo). Representa uma propriedade ou característica da entidade.
- **Domínio**: Conjunto de valores possíveis para um atributo. Por exemplo, o domínio do atributo "Data de Nascimento" seria o conjunto de todas as datas válidas.
- A força do modelo relacional está na forma como as tabelas se conectam entre si

EXEMPLO

- Cada linha representa uma **tupla**.
- Cada coluna representa um **atributo**.

Atributos

Relação

Tuplas

Aluno	Matrícula	Nome	Telefone	Curso	Idade
	1678	Alex	99 9999 9999	COMPUTAÇÃO	18
	1498	Marc	88 8888 8888	PSICOLOGIA	20
	1258	Antony	77 7777 7777	MEDICINA	23
	1335	Mariana	66 6666 6666	ODONTOLOGIA	22
	1256	John	55 5555 5555	DIREITO	19

ATRIBUTO CHAVE

- **Atributo Chave (Identificador)**
  - É um atributo que identifica unicamente cada instância de um entidade. Os valores do atributo chave são únicos.
  - Exemplo: Nº do protuário do Paciente, CPF e Chassi do Veículo
- **Chave Composta**
  - A chave pode ser formada por mais de um atributo.
  - Neste caso, a combinação dos valores dos atributos é que torna única cada instância da entidade.
  - Ex.: Os atributos CPF_PACIENTE + CRM_MÉDICO + DATA_CONSULTA identificam unicamente instância da entidade Consulta.

SUPERCHAVE

- Conjunto de atributos (colunas) que pode identificar unicamente cada linha (tupla) em uma tabela.
- Pode conter colunas que, sozinhas, não seriam capazes de identificar uma linha de forma única, mas que, combinadas, garantem a exclusividade dos dados.
- Uma super chave pode conter mais atributos do que o necessário para a identificação única.
- Ex.: Se "ID do Funcionário" é único, então "ID do Funcionário" + "Nome" também é uma super chave,

CHAVE CANDIDATA

- É uma super chave mínima, ou seja, uma super chave da qual nenhum atributo pode ser removido sem que ela perca sua propriedade de identificação única.
- Ela precisa ter valores únicos e não nulos.
- Cada tabela pode ter uma ou mais chaves candidatas, e uma delas é escolhida para ser a chave primária as outras se tornam chaves alternativas.

Aluno = {Nome, Idade, Curso, Matrícula, CPF}:

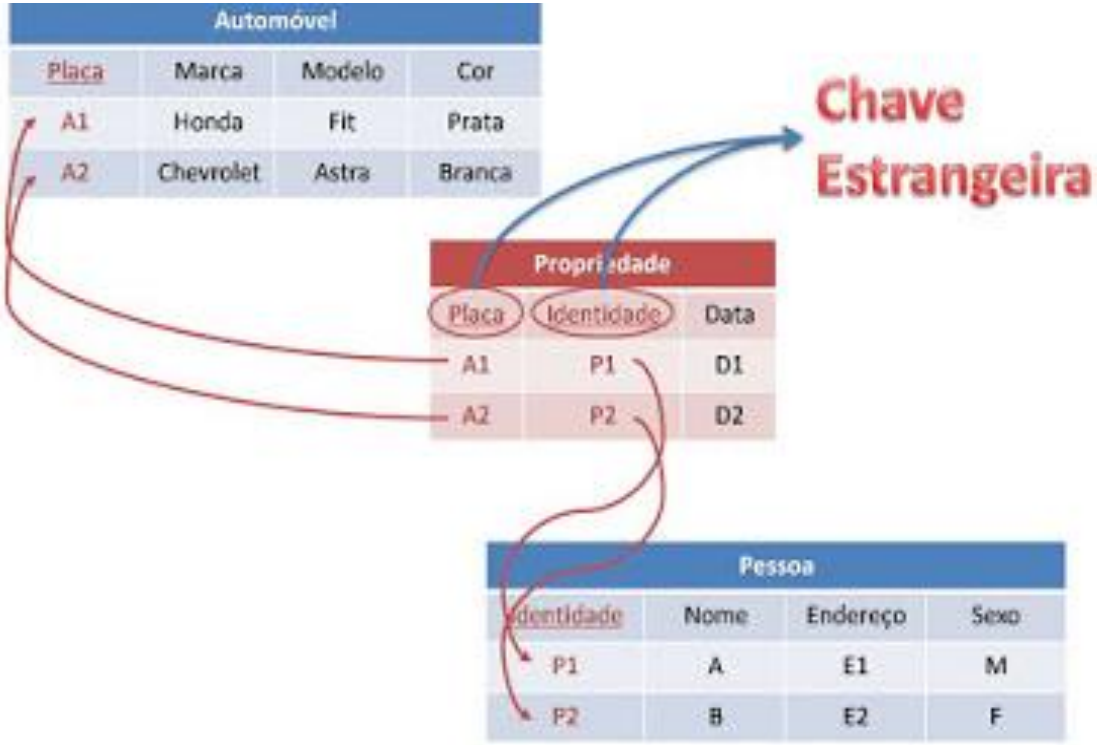
- CH1(Aluno) = {Matrícula}
- CH2(Aluno) = {CPF}

CHAVE PRIMÁRIA

- É considerado a chave candidata que foi escolhida no projeto do BD para identificar unicamente as instâncias de determinada entidade. Sendo a mais utilizada para busca de informações em relação à entidade.
- Caracteríticas
  - **Unicidade:** Cada valor na chave primária deve ser diferente para cada registro na tabela.
  - **Não Nulo:** Os campos da chave primária não podem conter valores nulos.
  - **Integridade dos Dados:** Ajuda a manter a precisão e a consistência dos dados no banco de dados.
  - **Identificador Único:** Funciona como a "identidade" de cada registro, permitindo recuperá-lo, atualizá-lo ou excluí-lo com facilidade.
  - **Referência:** estabelece relações entre tabelas, sendo referenciada por chaves estrangeiras em outras tabelas.
- Exemplos:
  - Um número de matricula de um professor em uma tabela de docentes .
  - O número de um CPF ou um ID de cliente em uma tabela de clientes.

CHAVE ESTRANGEIRA

- Atributo (campo) de uma tabela que se conecta à chave primária de outra tabela, criando um relacionamento entre elas.
- Assegura que os dados em uma tabela sejam consistentes com os dados em outra tabela relacionada
- É uma forma de "apontar" de um local para outro, conectando dados de maneira estruturada



VANTAGENS

- Simplicidade e clareza.
- Flexibilidade para consultas.
- Facilidade de manutenção.
- Suporte por diversos SGBDs (MySQL, PostgreSQL, SQL Server, etc.).
- O modelo relacional é a base teórica dos bancos de dados modernos.
- Compreender seus conceitos é essencial para projetar sistemas eficientes.
- Prática com SQL e modelagem é fundamental para o domínio da área.

EXEMPLO: BD DE UMA LOJA

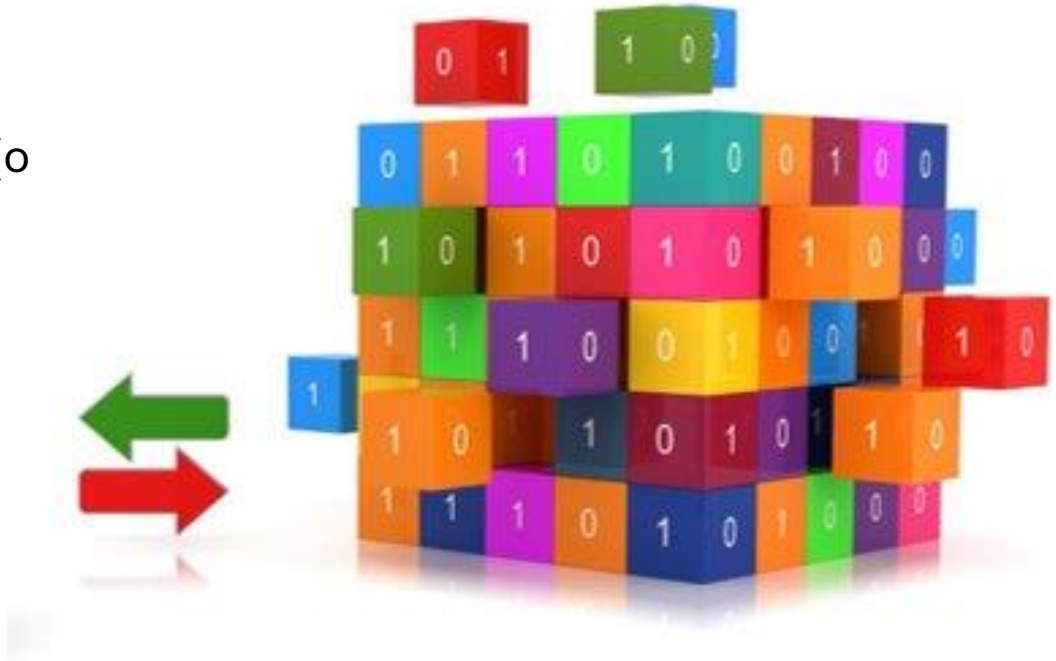
- Tabela CLIENTES: Armazena dados únicos dos clientes.
  - id_cliente (Chave Primária)
  - Nome
  - endereco
- Tabela PEDIDOS: Registra cada pedido.
  - id_pedido (Chave Primária)
  - data_pedido
  - id_cliente (Chave Estrangeira, referenciando a tabela CLIENTES)
- Tabela ITENS_PEDIDO: Lista os itens de cada pedido, resolvendo o relacionamento de muitos-para-muitos entre PEDIDOS e PRODUTOS.
  - id_item (Chave Primária)
  - id_pedido (Chave Estrangeira)
  - id_produto (Chave Estrangeira)
  - quantidade
- Tabela PRODUTOS: Contém os detalhes dos produtos.
  - id_produto (Chave Primária)
  - nome_produto
  - preco

DEPENDÊNCIA FUNCIONAL

Conceitos essenciais para integridade e modelagem eficaz

O QUE É DEPENDÊNCIA FUNCIONAL?

- **Dependência funcional (DF)** é uma relação entre dois conjuntos de atributos em uma tabela de banco de dados relacional.
- Ela especifica que o valor de um conjunto de atributos (o **determinante**) determina o valor de outro conjunto de atributos (o **dependente**).
- Exemplo: Em uma tabela de **Pedidos**, o número do pedido determina o prazo de entrega (Num_Pedido → Prazo_Entrega).



REPRESENTAÇÃO FORMAL

- Seja R uma relação (tabela) e X, Y subconjuntos de atributos de R.
- Um conjunto de atributos **X** determina funcionalmente um conjunto **Y** (**X** → **Y**) se, para cada valor de **X**, existe exatamente um valor de **Y**.
- Ou seja, Y é funcionalmente dependente de X se cada valor de X está associado unicamente a um valor de Y.

$$\mathbf{X} \rightarrow \mathbf{Y}$$

  - Lê-se: "X determina funcionalmente Y" ou "Y depende funcionalmente de X"
  - X: Determinante (Conjunto de atributos à esquerda)
  - Y: Dependente (Conjunto de atributos à direita)

EXEMPLO PRÁTICO

- Relação: **ALUNO**

Matricula	Nome	Curso	Nome_Orientador
101	João	ADS	Nathan
102	Ana	ADS	Paulo Filho
103	Pedro	Marketing	Jéssica

- Chave Primária:
  - **Matricula --> {Nome, Curso, Nome_Orientador}**
- Dependência Funcional Trivial ou Parcial:
  - **Matricula --> Nome**

TIPOS DE DF

- **Dependência Funcional Completa (DF Completa):**
- O atributo dependente Y é determinado por **todo** o conjunto de atributos determinante X, e não por uma parte dele.
  - Exemplo: Em uma tabela de **ITEM_PEDIDO**(Num_Pedido, Cod_Produto, Descricao_Produto, Preco_Unitario), se a chave é {Num_Pedido, Cod_Produto}, a DF:
  - {Num_Pedido, Cod_Produto} → {Preco_Unitario}

  é **completa** se o Preco_Unitario depende da combinação do pedido e do produto.

TIPOS DE DF

- **Dependência Funcional Parcial (DF Parcial):**
- O atributo dependente Y é determinado por um subconjunto próprio do determinante X que é uma chave candidata.
- Exemplo : Na mesma tabela acima, se:
  - {Cod_Produto} ⟶ {Descricao_Produto}
  - Isto é uma DF parcial, pois Descricao_Produto não precisa do Num_Pedido, apenas de Cod_Produto.

TIPOS DE DF

- **Dependência Funcional Transitiva (DF Transitiva):**
- Ocorre quando um atributo não-chave depende de outro atributo não-chave, que por sua vez depende da chave primária. Isso significa que há uma "cadeia" de dependência: a chave primária determina um atributo B, e esse atributo B determina outro atributo C

Matricula	Nome	Cod_Depto	Nome_Depto
101	Tainara	DTI	Diretoria de Tecnologia da Informação

  - O atributo não chave **Nome_Depto** é dependente de outro atributo não chave **Cod_Depto**, e não diretamente da chave principal **Matricula**.

IMPORTÂNCIA

- **Eliminação de Redundância**: DFs parciais e transitivas levam à repetição desnecessária de dados
- **Prevenção de Anomalias**: Evita problemas em inserção, exclusão e atualização de dados.
- **Normalização**: A identificação correta das DFs é o primeiro e mais importante passo para aplicar as Formas Normais (1FN, 2FN, 3FN).
- Ajudam no projeto de esquemas relacionais mais eficientes.

EXERCICIO: TABELA - PROJETO_FUNCIONARIO

CodFunc	NomeFunc	SalarioFunc	CodProjeto	NomeProjeto	OrcamentoProjeto	HorasAlocadas
F001	Ana Silva	5000	P101	Desenvolvimento	100000	40
F001	Ana Silva	5000	P102	Testes	50000	20
F002	Bruno Costa	6000	P101	Desenvolvimento	100000	30
F003	Carla Lima	4500	P103	Manutenção	75000	35

1. Qual é o conjunto mínimo de atributos (chave primária) que identifica unicamente cada linha da tabela PROJETO_FUNCIONARIO?
2. Liste todas as Dependências Funcionais (DF) e as classifique (Completa, Parcial ou Transitiva).